SVM Text Classification

1. Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
```

Purpose

TfidfVectorizer

Converts text into numerical TF-IDF vectors.

SVC

Implements the Support Vector Machine classifier.

Evaluation Metrics

Used to measure model performance:

```
accuracy_score()
precision_score()
recall_score()
f1_score()
```

2. Training Dataset

```
documents = [
    "machine learning improves prediction",
    "deep learning uses neural networks",
    "artificial intelligence is powerful",
    "football is a popular sport",
    "cricket is played worldwide",
    "soccer is an exciting game"
]
```

Training data contains two categories.

Technology Documents

```
machine learning improves prediction
deep learning uses neural networks
artificial intelligence is powerful
```

Sports Documents

```
football is a popular sport
cricket is played worldwide
soccer is an exciting game
```

3. Define Labels

```
labels = [
    "Technology",
    "Technology",
    "Technology",
    "Sports",
    "Sports",
    "Sports"
]
```

Each document is assigned a category.

Document	Category
machine learning improves prediction	Technology
deep learning uses neural networks	Technology
artificial intelligence is powerful	Technology
football is a popular sport	Sports
cricket is played worldwide	Sports
soccer is an exciting game	Sports

4. TF-IDF Feature Extraction

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

Why TF-IDF?

Machine learning algorithms require numerical input.

TF-IDF converts text into weighted numerical vectors.

Formula

Term Frequency (TF)

Inverse Document Frequency (IDF)

TF-IDF

Important words like:

```
machine
neural
intelligence
football
cricket
```

receive higher weights.

5. Create SVM Classifier

```
model = SVC(kernel='linear')
```

What is SVM?

Support Vector Machine is a supervised machine learning algorithm that finds the **best boundary (hyperplane)** separating classes.

Example:

```
Technology      |      Sports
--------------------------------
machine         | football
neural          | cricket
intelligence    | soccer
```

SVM tries to create a line (or hyperplane) that maximizes the separation between categories.

The parameter:

```
kernel='linear'
```

means a straight-line decision boundary is used.

6. Train the Model

```
model.fit(X, labels)
```

What Happens?

The model learns patterns that distinguish:

Technology

```
machine
learning
deep
neural
intelligence
```

from

Sports

```
football
cricket
soccer
sport
game
```

After training, the SVM can classify unseen text.

7. Test Documents

```
test_docs = [
    "neural network and machine learning",
    "football world cup"
]
```

Two new documents are provided.

Document 1

```
neural network and machine learning
```

Expected category:

```
Technology
```

Document 2

```
football world cup
```

Expected category:

```
Sports
```

8. Actual Labels

```
actual_labels = [
    "Technology",
    "Sports"
]
```

These are the correct answers used for evaluation.

9. Transform Test Documents

```
X_test = vectorizer.transform(test_docs)
```

The same TF-IDF transformation learned during training is applied to test data.

Example:

```
neural network and machine learning
```

↓

```
Numerical TF-IDF Vector
```

10. Make Predictions

```
predicted_labels = model.predict(X_test)
```

The SVM predicts the category for each document.

Expected results:

```
neural network and machine learning -> Technology

football world cup -> Sports
```

11. Display Predictions

```
for doc, pred in zip(test_docs, predicted_labels):
    print(doc, "->", pred)
```

Possible Output:

```
Predictions:

neural network and machine learning -> Technology
football world cup -> Sports
```

12. Model Evaluation

The model performance is measured using four metrics.

Accuracy

```
accuracy_score(actual_labels,
               predicted_labels)
```

Formula

Example:

```
2 Correct Predictions
2 Total Predictions
```

Measures:

> Out of all predicted classes, how many were correct?

Higher precision means fewer false positives.

Recall

```
recall_score(...,
             average='weighted')
```

Formula

Measures:

> Out of all actual class instances, how many were identified correctly?

Higher recall means fewer missed predictions.

F1 Score

```
f1_score(...,
         average='weighted')
```

Formula

F1 balances precision and recall.

13. Display Evaluation Results

```
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

If both test documents are correctly classified, the output may be:

```
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

Why SVM is Popular for Text Classification

Advantages

- ✅ Works well with high-dimensional text data
- ✅ Handles sparse TF-IDF vectors effectively
- ✅ Good accuracy on small and medium datasets
- ✅ Strong classification performance

Disadvantages

❌ Training can become slow on very large datasets

❌ Less interpretable than some simpler algorithms

Program Workflow

```
Training Text
      ↓
TF-IDF Vectorization
      ↓
SVM Training
      ↓
Learn Decision Boundary
      ↓
New Documents
       ↓
 TF-IDF Transformation
       ↓
 Prediction
       ↓
 Accuracy / Precision /
 Recall / F1 Evaluation
```

Difference: Naive Bayes vs SVM

Feature	Naive Bayes	SVM
Method	Probabilistic	Margin-based
Speed	Faster	Slightly slower
Accuracy	Good	Often better
High-dimensional data	Good	Excellent
Text Classification	Popular	Very Popular

Key NLP Concepts Used

Concept	Purpose
TF-IDF	Convert text to weighted numeric vectors
SVM	Classify text into categories
Supervised Learning	Learn from labeled examples
Text Classification	Assign categories to documents
Evaluation Metrics	Measure model performance

Sample Interview Questions

Q1. Why is TF-IDF used before SVM?

Answer: SVM works with numerical features, so TF-IDF converts text into weighted vectors while preserving important terms.

Q2. What does kernel='linear' mean?

Answer: It tells SVM to use a linear decision boundary (hyperplane) to separate classes.

Q3. Why is SVM effective for text classification?

Answer: Text data is usually high-dimensional and sparse. SVM handles such data efficiently and often provides high classification accuracy.

Q4. What is the expected output of the program?

Answer:

```
neural network and machine learning -> Technology
football world cup -> Sports

Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

assuming both test documents are correctly classified.

Key Takeaway

This program builds a **Text Classification System** using **TF-IDF** and **Support Vector Machine (SVM)**. It learns patterns from labeled Technology and Sports documents, predicts categories for new documents, and evaluates performance using **Accuracy, Precision, Recall, and F1-Score**.